REVISION CHEAT SHEET:

SQL Server Functions

COMPLETE INTERVIEW-READY NOTES - SCALAR UDF - INLINE TVF - MSTVF - APPLY - PERFORMANCE

TOPIC & SUBTOPIC INDEX

Study order: function basics -> built-in vs UDF -> scalar UDFs -> scalar UDF inlining -> inline TVFs and APPLY -> MSTVFs -> limitations/ determinism/schemabinding -> metadata/security/performance -> practical examples -> interview revision.

What Is a Function?	Basic Function Idea	Why Use Functions?
Example Without a Function	Main SQL Server Function Categories	Built-In Functions
Fundraising Examples of Built-In Functions	User-Defined Functions - UDFs	Function vs Stored Procedure
Function Parameters	Function Return Values	CREATE FUNCTION
ALTER FUNCTION	CREATE OR ALTER FUNCTION	DROP FUNCTION
Calling a Scalar Function	Scalar-Valued Function	Real Fundraising Scalar Function
Scalar Function in SELECT	Scalar Function in WHERE	Scalar UDF that Reads a Table
Scalar UDF Performance	Scalar UDF Inlining Concept	Checking Scalar UDF Inlineability
INLINE Option	Scalar UDF Best Practice	Table-Valued Functions
Inline Table-Valued Function - iTVF	Real Fundraising iTVF	Inline TVF with Multiple Parameters
Inline TVF with JOIN	iTVF with CROSS APPLY	OUTER APPLY
CROSS APPLY vs OUTER APPLY	iTVF vs View	Inline TVF Performance
Multi-Statement Table-Valued Function - MSTVF	Real Fundraising MSTVF Example	Table Variable in MSTVF
Does This Example Really Need an MSTVF?	MSTVF Performance Considerations	Practical MSTVF Rule
Inline TVF vs MSTVF	Function Limitations	Deterministic Functions
Nondeterministic Functions	Determinism and Indexing	WITH SCHEMABINDING
Functions and NULL	RETURNS NULL ON NULL INPUT	Function Dependencies
Function Metadata	Function Permissions	Nested Function Calls
Built-In Functions vs User-Defined Functions	Scalar UDF vs Inline TVF	Function vs View
Function vs Stored Procedure	Scalar Function vs Computed Expression	Functions and Performance
Function Performance Best Practices	Practical Example 1 - Net Gift Calculation	Practical Example 2 - Constituent Gifts iTVF
Practical Example 5 - Constituents and TVF with APPLY	Performance Rewrite of the Previous Example	Practical Example 7 - Giving Summary iTVF
Common Function Mistakes	Choosing the Correct Function Type	Master Function Comparison
25 Interview Questions and Practical Problems	25. Full Interview Problem	Understand the Full Example
Final Revision Map	Final Easy Memory Rules	

A **function** in SQL Server is a reusable piece of logic that accepts zero or more input values and returns a value.

Depending on the type of function, it can return:

EXAMPLE / FLOW

```
One scalar value

or
```

```
A table of rows and columns
```

Functions are especially useful when the same calculation, filtering rule, or parameterized table query needs to be reused in several SQL statements.

Throughout this chapter, examples use your `FundraisingDb` tables:

EXAMPLE / FLOW

```
core.Tenant
core.AppUser

constituent.Constituent
constituent.Address
constituent.ConstituentAddress

fundraising.Campaign
fundraising.Fund
fundraising.Gift
fundraising.GiftAllocation

engagement.Interaction
engagement.Task
```

The central fundraising relationship remains:

EXAMPLE / FLOW

```
Constituent
     |
     | makes
     v
Gift
     |
     +----------> Campaign
     |
     v
GiftAllocation
     |
     v
Fund
```

What Is a Function?

Simple Definition

A SQL Server function is reusable database logic that returns something.

That "something" can be:

EXAMPLE / FLOW

```
Single value
```

such as:

EXAMPLE / FLOW

```
$1,000 gift
minus
$25 processing fee
=
$975 net gift
```

or:

EXAMPLE / FLOW

```
Table result
```

such as:

EXAMPLE / FLOW

```
All Gifts belonging to Constituent 1001
```

Basic Function Idea

EXAMPLE / FLOW

```
Input
  |
  v
Function
  |
  v
Returned value
```

Example:

EXAMPLE / FLOW

```
Total Gift Amount = 1000
Fee Amount        = 25

          |
          v

CalculateNetGiftAmount()

          |
          v

975
```

Why Use Functions?

Functions can help with:

EXAMPLE / FLOW

```
Reusable calculations

Centralizing repeated expressions

Returning parameterized table results

Simplifying queries

Using reusable logic inside SELECT

Using reusable logic inside WHERE

Using reusable table logic in JOIN / APPLY

Supporting computed expressions

Making business calculations easier to reuse
```

Example Without a Function

Suppose several reports need:

EXAMPLE / FLOW

```
Net Gift Amount
=
TotalAmount - FeeAmount
```

Without a function, the calculation might be repeated everywhere:

SQL

```sql
SELECT
    GiftId,
    TotalAmount,
    TotalAmount - 25 AS NetAmount
FROM fundraising.Gift;
```

Another query:

SQL

```sql
SELECT
    GiftId
FROM fundraising.Gift
WHERE TotalAmount - 25 >= 1000;
```

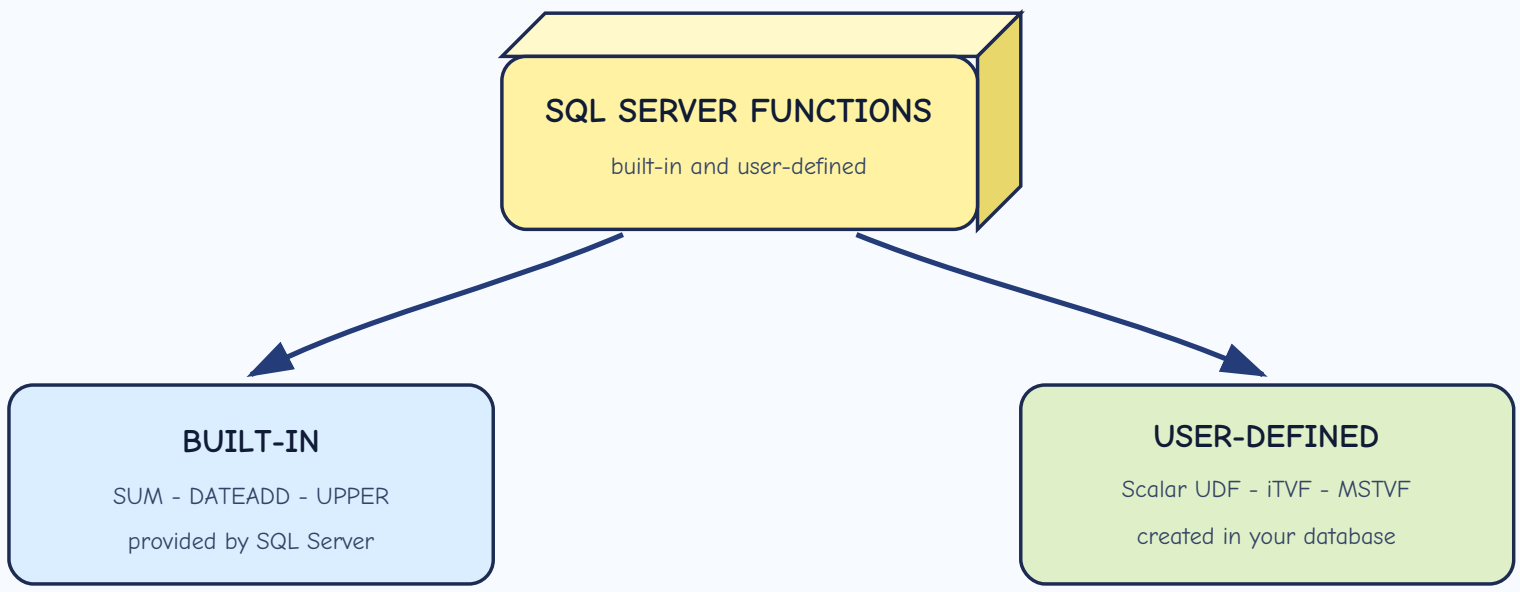
If the calculation becomes more complicated, repeating it everywhere becomes harder to maintain.

A scalar function can sometimes centralize that logic.

Main SQL Server Function Categories

3D DIAGRAM

SQL Server Function Families



Choose the function family from the shape of the required result and the operation you need.

At a high level:

EXAMPLE / FLOW

```
SQL Server Functions
        |
```

```
+---- Built-in Functions
|
+---- User-Defined Functions
          |
          +---- Scalar UDF
          |
          +---- Inline Table-Valued Function
          |
          +---- Multi-Statement
               Table-Valued Function
```

SQL Server documentation separates system/built-in functions from user-defined scalar and table-valued functions.

Built-In Functions

Simple Definition

A **built-in function** is already provided by SQL Server.

You do not create it yourself.

Examples include:

EXAMPLE / FLOW

```
COUNT()

SUM()

AVG()

MAX()

MIN()

GETDATE()

DATEADD()

DATEDIFF()

UPPER()

LOWER()

LEN()

COALESCE()

ISNULL()

ROW_NUMBER()
```

Fundraising Examples of Built-In Functions

Total Gifts:

SQL

```sql
SELECT
    SUM(TotalAmount) AS TotalGiving
FROM fundraising.Gift;
```

Average Gift:

```sql
SELECT
    AVG(TotalAmount) AS AverageGift
FROM fundraising.Gift;
```

Largest Gift:

```sql
SELECT
    MAX(TotalAmount) AS LargestGift
FROM fundraising.Gift;
```

Uppercase donor name:

```sql
SELECT
    ConstituentId,
    UPPER(DisplayName) AS DisplayNameUpper
FROM constituent.Constituent;
```

Add 30 days to a Task due date:

```sql
SELECT
    TaskId,
    DueDate,
    DATEADD(DAY, 30, DueDate) AS ExtendedDueDate
FROM engagement.Task;
```

User-Defined Functions - UDFs

Simple Definition

A **User-Defined Function**, usually called a **UDF**, is a function that you create inside the database.

Main T-SQL UDF forms:

```
Scalar-valued function

Inline table-valued function

Multi-statement table-valued function
```

Function vs Stored Procedure

This is one of the most important distinctions.

A function is designed to:

```
Return a value
or
return a table
```

A stored procedure is designed to:

```
Perform an operation
```

Simple Comparison

EXAMPLE / FLOW

```
Function
-> "Calculate or return something."
```

EXAMPLE / FLOW

```
Stored Procedure
-> "Execute a database operation."
```

Function vs Stored Procedure Table

Function	Stored Procedure
Returns scalar or table	Can return result sets, output parameters, status
Can be used inside queries	Executed with `EXEC`
Can appear in `SELECT`	Cannot normally be called inside `SELECT`
Can appear in `FROM` if TVF	Procedure cannot act like a table
More restrictive	Much more flexible
Cannot normally modify database state	Can INSERT/UPDATE/DELETE
No dynamic SQL	Dynamic SQL allowed
No temp tables	Temp tables allowed
No `TRY...CATCH`	`TRY...CATCH` allowed
Cannot call normal stored procedures	Can call other procedures
Best for reusable query/calculation logic	Best for operations/workflows

SQL Server UDFs cannot modify database state, use dynamic SQL or temp tables, call ordinary stored procedures, or use `TRY...CATCH`; table variables are allowed.

Function Parameters

Functions can accept input parameters.

Example:

EXAMPLE / FLOW

```
Gift Amount = 1000
Fee Amount = 25
```

Function:

EXAMPLE / FLOW

```
ufn_CalculateNetGiftAmount
```

Parameters:

EXAMPLE / FLOW

```
@TotalAmount

@FeeAmount
```

Returned value:

EXAMPLE / FLOW

```
975
```

Important Function Parameter Rule

SQL Server UDF parameters are primarily input values.

Functions do not use stored-procedure-style:

EXAMPLE / FLOW

```
OUTPUT parameters
```

to return multiple values.

Instead:

EXAMPLE / FLOW

```
Scalar UDF
-> returns one scalar value

TVF
-> returns a table
```

Function Return Values

A function must define what it returns.

Scalar function:

SQL

```
RETURNS DECIMAL(18,2)
```

TVF:

SQL

```
RETURNS TABLE
```

or:

SQL

```
RETURNS @Result TABLE
(
    ...
)
```

CREATE FUNCTION

Basic scalar syntax:

SQL

```
CREATE FUNCTION schema_name.function_name
(
    @Parameter DataType
)
RETURNS ReturnDataType
AS
BEGIN

    RETURN expression;

END;
GO
```

ALTER FUNCTION

`ALTER FUNCTION` changes an existing function.

Example:

```sql
ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
AS
BEGIN
    RETURN
        @TotalAmount
        -
        COALESCE(@FeeAmount, 0);
END;
GO
```

Important: `ALTER FUNCTION` cannot change a scalar function into a table-valued function, nor convert an inline TVF into a multi-statement TVF or vice versa.

CREATE OR ALTER FUNCTION

Modern SQL Server supports:

```sql
CREATE OR ALTER FUNCTION
```

This is convenient in deployment scripts.

If the function does not exist:

```
Create it
```

If it exists:

```
Alter it
```

DROP FUNCTION

```sql
DROP FUNCTION fundraising.ufn_CalculateNetGiftAmount;
GO
```

This removes the function definition.

It does not remove:

```
Gift

Constituent

Campaign

Fund
```

data.

Calling a Scalar Function

Scalar functions should normally be called with at least their two-part name:

EXAMPLE / FLOW

```
schema.function
```

Example:

SQL

```
SELECT
    fundraising.ufn_CalculateNetGiftAmount
    (
        1000,
        25
    );
```

Microsoft specifically documents the need to schema-qualify scalar UDF calls.

Scalar-Valued Function

Simple Definition

A **scalar-valued function** returns one single value.

Examples:

EXAMPLE / FLOW

```
Number

Date

String

Boolean-like BIT

Decimal amount
```

Real Fundraising Scalar Function

Suppose we want:

EXAMPLE / FLOW

```
Net Gift Amount
=
Total Gift Amount - Processing Fee
```

Create:

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
WITH SCHEMABINDING
AS
BEGIN

    RETURN
        @TotalAmount
```

```
        -
        COALESCE(@FeeAmount, 0);

END;
GO
```

Understand the Function

Input:

EXAMPLE / FLOW

```
@TotalAmount = 1000

@FeeAmount = 25
```

Calculation:

EXAMPLE / FLOW

```
1000 - 25
```

Return:

EXAMPLE / FLOW

```
975
```

Calling Scalar Function with Parameters

SQL

```
SELECT
    fundraising.ufn_CalculateNetGiftAmount
    (
        1000,
        25
    ) AS NetGiftAmount;
```

Result:

EXAMPLE / FLOW

```
975.00
```

Scalar Function in SELECT

Suppose a query has some processing fee value available.

For learning purposes:

SQL

```
SELECT
    GiftId,
    TotalAmount,

    fundraising.ufn_CalculateNetGiftAmount
    (
        TotalAmount,
        25
    ) AS NetGiftAmount

FROM fundraising.Gift;
```

Conceptually:

EXAMPLE / FLOW

```
Gift 10000
Total = 1000
Fee   = 25

Function returns
975
```

Scalar Function in an Expression

You can combine a scalar UDF with other expressions.

Example:

SQL

```sql
SELECT
    GiftId,

    fundraising.ufn_CalculateNetGiftAmount
    (
        TotalAmount,
        25
    )
    * 0.10 AS TenPercentOfNetAmount

FROM fundraising.Gift;
```

Scalar Function in WHERE

A scalar UDF can also be used in filtering.

Example:

SQL

```sql
SELECT
    GiftId,
    TotalAmount

FROM fundraising.Gift

WHERE fundraising.ufn_CalculateNetGiftAmount
      (
          TotalAmount,
          25
      ) >= 1000;
```

This is valid.

But there is an important performance warning.

Scalar Function in WHERE - Performance Warning

Functions around a column can sometimes make it harder for SQL Server to use an index efficiently.

Example:

SQL

```sql
WHERE fundraising.ufn_CalculateNetGiftAmount
      (
          TotalAmount,
          25
      ) >= 1000
```

may be less optimizer-friendly than expressing a simple equivalent predicate directly when possible.

For this particular calculation:

EXAMPLE / FLOW

```
TotalAmount - 25 >= 1000
```

means:

EXAMPLE / FLOW

```
TotalAmount >= 1025
```

A direct predicate may allow a more efficient index seek.

Therefore:

> Do not put scalar functions around search columns automatically.

Scalar UDF that Reads a Table

A scalar function can query tables.

Example:

> Return a Constituent's lifetime giving.

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_GetLifetimeGiving
(
    @ConstituentId BIGINT
)
RETURNS DECIMAL(18,2)
AS
BEGIN

    DECLARE
        @LifetimeGiving DECIMAL(18,2);

    SELECT
        @LifetimeGiving =
            COALESCE
            (
                SUM(TotalAmount),
                0
            )

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId;

    RETURN @LifetimeGiving;

END;
GO
```

Calling the Lifetime Giving Function

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    fundraising.ufn_GetLifetimeGiving
```

```
    (
        c.ConstituentId
    ) AS LifetimeGiving

FROM constituent.Constituent AS c;
```

Business meaning:

EXAMPLE / FLOW

```
For each Constituent
    |
    v
Pass ConstituentId to function
    |
    v
Function sums that Constituent's Gifts
    |
    v
Return lifetime total
```

Is This Always a Good Design?

No.

This is an important performance lesson.

If:

EXAMPLE / FLOW

```
1,000,000 Constituents
```

are returned, logically the function may need to calculate giving repeatedly.

A set-based query is often better:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,
    COALESCE(SUM(g.TotalAmount), 0)
        AS LifetimeGiving

FROM constituent.Constituent AS c

LEFT JOIN fundraising.Gift AS g
    ON g.ConstituentId =
       c.ConstituentId

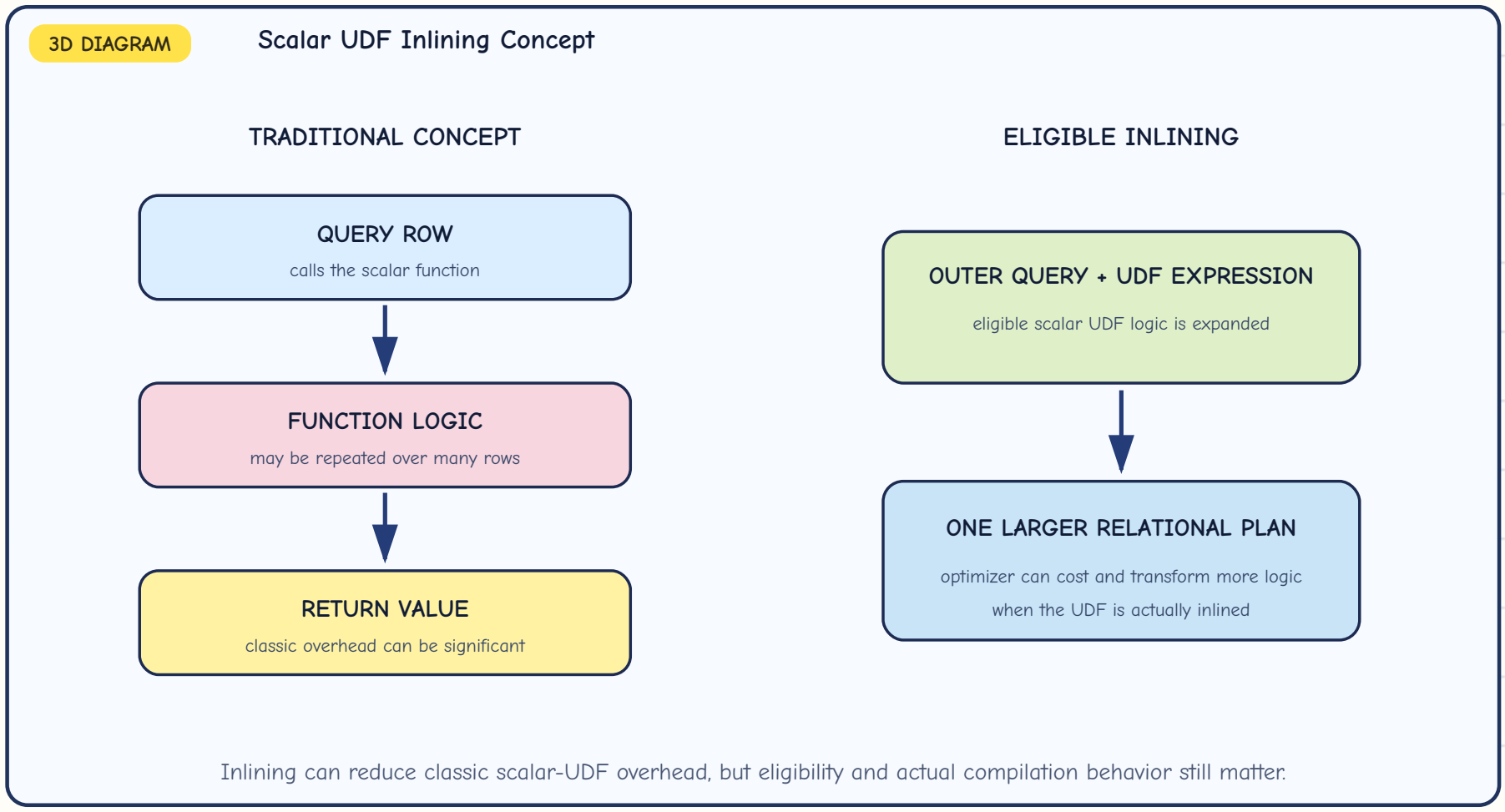
GROUP BY
    c.ConstituentId,
    c.DisplayName;
```

Scalar UDF Performance

Historically, T-SQL scalar UDFs were often expensive because their operations could be evaluated iteratively and inhibit query optimization and parallelism.

SQL Server introduced **Scalar UDF Inlining** starting with SQL Server 2019 compatibility level 150. Eligible scalar functions can be transformed into relational expressions inside the calling query, allowing the optimizer to cost and optimize the logic more effectively. Inlining is enabled by default at compatibility level 150 or higher, but not every scalar UDF is eligible and eligibility does not guarantee that every call is actually inlined.

Scalar UDF Inlining Concept



Old conceptual behavior:

EXAMPLE / FLOW

```
Query row
   |
   v
Call function
   |
   v
Execute function logic
   |
   v
Return value
```

Repeated many times.

With eligible inlining:

EXAMPLE / FLOW

```
Outer Query
   +
Function expression
   |
   v
Optimizer sees larger relational expression
   |
   v
One optimized plan
```

Checking Scalar UDF Inlineability

`sys.sql_modules` contains an:

EXAMPLE / FLOW

`is_inlineable`

property for scalar UDFs.

Example:

SQL

```
SELECT
    SCHEMA_NAME(o.schema_id)
        AS SchemaName,

    o.name AS FunctionName,

    sm.is_inlineable

FROM sys.sql_modules AS sm

INNER JOIN sys.objects AS o
    ON o.object_id =
       sm.object_id

WHERE o.type = 'FN';
```

INLINE Option

A scalar function can specify:

SQL

```
WITH INLINE = ON
```

or:

SQL

```
WITH INLINE = OFF
```

Example:

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
WITH
    SCHEMABINDING,
    INLINE = ON
AS
BEGIN

    RETURN
        @TotalAmount
        -
        COALESCE(@FeeAmount, 0);

END;
GO
```

If `INLINE = ON` is requested but SQL Server determines the function is not eligible for inlining, creation/alteration can fail.

Scalar UDF Best Practice

Prefer a scalar UDF when:

EXAMPLE / FLOW

```
The calculation is genuinely reusable

The calculation is reasonably small

It improves maintainability

Inlining is possible where performance matters

The logic cannot be expressed more naturally
as a normal set-based query
```

Avoid using a scalar UDF as a replacement for every repeated expression.

Table-Valued Functions

Simple Definition

A **table-valued function**, usually called a **TVF**, returns a table instead of one value.

Main types:

EXAMPLE / FLOW

```
Inline TVF

Multi-Statement TVF
```

TVF Mental Model

Scalar function:

EXAMPLE / FLOW

```
Inputs
  |
  v
Function
  |
  v
One value
```

Table-valued function:

EXAMPLE / FLOW

```
Inputs
  |
  v
Function
  |
  v
Rows + Columns
```

Inline Table-Valued Function - iTVF

Simple Definition

An **inline table-valued function** returns the result of one SELECT statement.

It is conceptually similar to:

EXAMPLE / FLOW

```
a parameterized view
```

An inline TVF has no return table variable; the table shape comes directly from the single SELECT.

Inline TVF Syntax

SQL

```
CREATE FUNCTION schema.function_name
(
    @Parameter DataType
)
RETURNS TABLE
AS
RETURN
(
    SELECT ...
);
GO
```

Real Fundraising iTVF

Requirement:

Return Gifts for one Constituent.

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGifts
(
    @ConstituentId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        ConstituentId,
        CampaignId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId
);
GO
```

Calling an Inline TVF

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount

FROM fundraising.ufn_ConstituentGifts
(
    1001
);
```

This behaves like a table source.

iTVF Flow

EXAMPLE / FLOW

```
@ConstituentId = 1001
        |
        v
ufn_ConstituentGifts
        |
        v
SELECT FROM Gift
WHERE ConstituentId = 1001
        |
        v
Table result
```

Inline TVF with Multiple Parameters

Requirement:

Return Gifts for one Constituent between two dates.

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGiftsByDate
(
    @ConstituentId BIGINT,
    @StartDate DATE,
    @EndDate DATE
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId

      AND GiftDate >=
          @StartDate

      AND GiftDate <
          DATEADD
          (
              DAY,
              1,
              @EndDate
          )
);
GO
```

Call:

SQL

```
SELECT *
FROM fundraising.ufn_ConstituentGiftsByDate
(
    1001,
    '2026-01-01',
```

```
    '2026-12-31'
);
```

Inline TVF with JOIN

A TVF's internal **SELECT** can join related tables.

Requirement:

> Return one Constituent's Gift allocation details.

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGiftAllocations
(
    @ConstituentId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        g.GiftId,
        g.GiftDate,
        g.TotalAmount,

        ga.GiftAllocationId,
        ga.FundId,
        ga.AllocationAmount,

        f.FundName

    FROM fundraising.Gift AS g

    INNER JOIN fundraising.GiftAllocation AS ga
        ON ga.GiftId =
           g.GiftId

    INNER JOIN fundraising.Fund AS f
        ON f.FundId =
           ga.FundId

    WHERE g.ConstituentId =
          @ConstituentId
);
GO
```

Querying the iTVF

SQL

```
SELECT
    GiftId,
    GiftDate,
    FundName,
    AllocationAmount

FROM fundraising.ufn_ConstituentGiftAllocations
(
    1001
);
```

iTVF with SELECT

This is the simplest use:

```sql
SELECT *
FROM fundraising.ufn_ConstituentGifts(1001);
```

iTVF with JOIN

Suppose we want campaign information too.

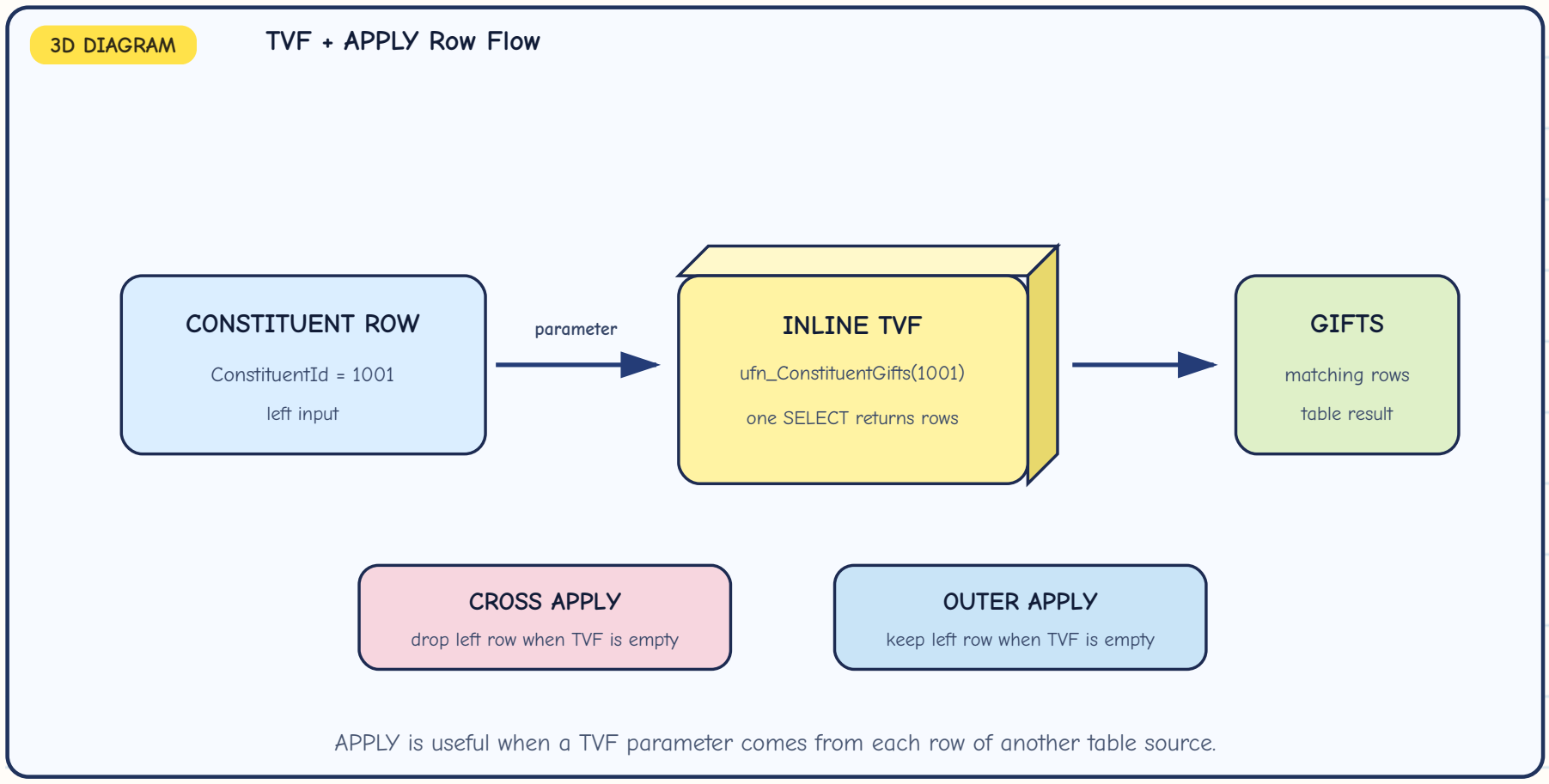
```sql
SELECT
    g.GiftId,
    g.GiftDate,
    g.TotalAmount,

    ca.CampaignName

FROM fundraising.ufn_ConstituentGifts
     (
         1001
     ) AS g

LEFT JOIN fundraising.Campaign AS ca
    ON ca.CampaignId =
       g.CampaignId;
```

iTVF with CROSS APPLY



APPLY becomes especially useful when the function parameter must come from another table row.

Suppose:

> For every Constituent, return their Gifts.

```sql
SELECT
    c.ConstituentId,
    c.DisplayName,

    g.GiftId,
    g.GiftDate,
    g.TotalAmount

FROM constituent.Constituent AS c

CROSS APPLY
    fundraising.ufn_ConstituentGifts
    (
        c.ConstituentId
    ) AS g;
```

APPLY evaluates the table-valued function using values from each row of the left input.

CROSS APPLY Flow

For:

```
Constituent 1001
```

SQL Server evaluates:

```
ufn_ConstituentGifts(1001)
```

Then:

```
Constituent 1002
```

uses:

```
ufn_ConstituentGifts(1002)
```

Conceptually:

```
Constituent
    |
    | ConstituentId
    v
TVF
    |
    v
Matching Gifts
```

CROSS APPLY Behavior

If the function returns no rows for one Constituent:

```
that Constituent disappears
```

from the result.

It behaves similarly to an inner relationship in that respect.

OUTER APPLY

If you want to keep Constituents even when the function returns no rows:

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    g.GiftId,
    g.TotalAmount

FROM constituent.Constituent AS c

OUTER APPLY
    fundraising.ufn_ConstituentGifts
    (
        c.ConstituentId
    ) AS g;
```

`OUTER APPLY` preserves the left-side row even when the right table source returns no rows.

CROSS APPLY vs OUTER APPLY

EXAMPLE / FLOW

```
CROSS APPLY
-> Return only left rows
  for which TVF returns rows
```

EXAMPLE / FLOW

```
OUTER APPLY
-> Keep every left row
  even when TVF returns nothing
```

Memory:

EXAMPLE / FLOW

```
CROSS APPLY
~ INNER behavior

OUTER APPLY
~ LEFT OUTER behavior
```

iTVF vs View

An inline TVF and a view can look similar.

View:

SQL

```
SELECT *
FROM fundraising.vw_Gifts;
```

iTVF:

SQL

```
SELECT *
FROM fundraising.ufn_ConstituentGifts
```

```
(
    1001
);
```

The important difference:

EXAMPLE / FLOW

```
View
-> No normal input parameters

iTVF
-> Accepts parameters
```

iTVF vs View Table

Inline TVF	View
Can accept parameters	No normal parameters
Returns table	Returns table-shaped result
Used in **FROM**	Used in **FROM**
Single SELECT body	SELECT definition
Excellent parameterized relational abstraction	Excellent reusable non-parameterized abstraction

Easy memory:

EXAMPLE / FLOW

```
iTVF
=
Parameterized view-like object
```

Inline TVF Performance

Inline TVFs are usually optimizer-friendly because SQL Server can expand the function's relational expression into the calling query.

There is no hidden return-table variable as there is with a multi-statement TVF.

For reusable parameterized relational logic:

Prefer an inline TVF when one **SELECT** can express the requirement.

Multi-Statement Table-Valued Function - MSTVF

Simple Definition

A **multi-statement table-valued function** returns a table, but unlike an iTVF, its body can contain multiple T-SQL statements.

It defines a table variable that collects rows before being returned.

MSTVF Syntax

SQL

```
CREATE FUNCTION schema.function_name
(
    @Parameter DataType
```

```
)
RETURNS @Result TABLE
(
    Column1 DataType,
    Column2 DataType
)
AS
BEGIN

    INSERT INTO @Result
    (
        Column1,
        Column2
    )
    SELECT
        ...;

    RETURN;

END;
GO
```

Real Fundraising MSTVF Example

Suppose we want a function returning summary metrics for one Constituent.

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGivingMetrics
(
    @ConstituentId BIGINT
)
RETURNS @Result TABLE
(
    MetricName VARCHAR(50) NOT NULL,
    MetricValue DECIMAL(18,2) NOT NULL
)
AS
BEGIN

    INSERT INTO @Result
    (
        MetricName,
        MetricValue
    )
    SELECT
        'LifetimeGiving',
        COALESCE
        (
            SUM(TotalAmount),
            0
        )

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId;

    INSERT INTO @Result
    (
        MetricName,
        MetricValue
    )
    SELECT
        'LargestGift',
```

```
        COALESCE
        (
            MAX(TotalAmount),
            0
        )

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId;

    RETURN;

END;
GO
```

Calling the MSTVF

SQL

```
SELECT
    MetricName,
    MetricValue

FROM fundraising.ufn_ConstituentGivingMetrics
(
    1001
);
```

Possible result:

MetricName	MetricValue
LifetimeGiving	50000
LargestGift	10000

What Happens Inside an MSTVF?

EXAMPLE / FLOW

```
Call function
   |
   v
Create internal return table variable
   |
   v
Statement 1 inserts rows
   |
   v
Statement 2 inserts rows
   |
   v
More processing if needed
   |
   v
RETURN
   |
   v
Caller receives table
```

Table Variable in MSTVF

This:

```sql
RETURNS @Result TABLE
(
    ...
)
```

creates the internal return table variable.

Inside the function you can:

```
INSERT into it

UPDATE it

DELETE from it
```

as local function processing.

UDFs cannot modify normal database tables, but local table variables can be modified.

Better Example - Multi-Step Logic

Suppose a function returns Gifts and labels large donations.

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGiftReport
(
    @ConstituentId BIGINT,
    @MajorGiftAmount DECIMAL(18,2)
)
RETURNS @Result TABLE
(
    GiftId BIGINT NOT NULL,
    GiftDate DATE NOT NULL,
    TotalAmount DECIMAL(18,2) NOT NULL,
    GiftCategory VARCHAR(20) NOT NULL
)
AS
BEGIN

    INSERT INTO @Result
    (
        GiftId,
        GiftDate,
        TotalAmount,
        GiftCategory
    )

    SELECT
        GiftId,
        GiftDate,
        TotalAmount,

        CASE
            WHEN TotalAmount >=
                 @MajorGiftAmount
                THEN 'Major'

            ELSE 'Standard'
        END
```

```
    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId;

    RETURN;

END;
GO
```

Does This Example Really Need an MSTVF?

No.

That is an important design lesson.

The same logic can be written as an iTVF:

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGiftReportInline
(
    @ConstituentId BIGINT,
    @MajorGiftAmount DECIMAL(18,2)
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        GiftDate,
        TotalAmount,

        CASE
            WHEN TotalAmount >=
                 @MajorGiftAmount
                THEN 'Major'

            ELSE 'Standard'
        END AS GiftCategory

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId
);
GO
```

Because one **SELECT** is enough:



Prefer the inline TVF.

MSTVF Performance Considerations

MSTVFs have historically been difficult for the optimizer because their result is stored in an internal table variable without normal base-table statistics.

Older behavior used fixed cardinality guesses. SQL Server 2017 and compatibility level 140 introduced **interleaved execution** for eligible MSTVF queries, which can execute the function during optimization and use its actual row count when optimizing the remaining plan. This improves some MSTVF scenarios but does not mean MSTVFs are always as optimizer-friendly as inline TVFs.

Practical MSTVF Rule

If the function can be written as:

EXAMPLE / FLOW

```
one SELECT
```

prefer:

EXAMPLE / FLOW

```
Inline TVF
```

If the result truly requires:

EXAMPLE / FLOW

```
multiple statements

staged table-variable processing

conditional population
```

then consider:

EXAMPLE / FLOW

```
MSTVF
```

but test performance carefully.

Inline TVF vs MSTVF

Inline TVF	MSTVF
One SELECT	Multiple statements
No return table variable	Uses return table variable
Usually optimizer-friendly	Harder cardinality estimation
Often better performance	Can be slower for large results
Simple relational logic	More procedural table-building logic

Function Limitations

SQL Server functions intentionally have restrictions.

A function is not meant to perform arbitrary database workflows.

Important limitations include:

EXAMPLE / FLOW

```
Cannot modify normal database state

Cannot INSERT/UPDATE/DELETE normal base tables

Cannot use dynamic SQL

Cannot use temporary tables

Cannot call ordinary stored procedures
```

```
Cannot use TRY...CATCH

Cannot use RAISERROR

Cannot return multiple result sets

Cannot use OUTPUT INTO targeting a table

SET statement options such as SET NOCOUNT ON
are not allowed
```

Table variables are allowed.

Why These Limitations Exist

A function should behave conceptually like:

EXAMPLE / FLOW

```
Inputs
  |
  v
Calculate / query
  |
  v
Return result
```

not:

EXAMPLE / FLOW

```
Receive input
  |
  v
Start transaction
  |
  v
Insert Gift
  |
  v
Send email
  |
  v
Update Fund
  |
  v
Return several result sets
```

The second pattern belongs much more naturally in:

EXAMPLE / FLOW

```
Stored procedure
```

or application/service logic.

Function Cannot Modify Gift Data

This kind of idea is not allowed in a normal UDF:

```sql
UPDATE fundraising.Gift
SET TotalAmount = 500
WHERE GiftId = 10000;
```

A UDF cannot use this to change persistent database state.

Use a:

```
stored procedure
```

for that operation.

Function Cannot Use #Temp Tables

This is not allowed inside a T-SQL UDF:

```sql
CREATE TABLE #GiftSummary
(
    ...
);
```

Use:

```
table variable
```

if function-local table storage is required.

Function Cannot Use Dynamic SQL

This is not allowed:

```sql
EXEC sys.sp_executesql @Sql;
```

inside an ordinary T-SQL UDF.

If the requirement genuinely needs dynamic SQL:

```
stored procedure
```

is a better fit.

Function Cannot Use TRY...CATCH

This is not allowed in a T-SQL UDF:

```sql
BEGIN TRY
    ...
END TRY
BEGIN CATCH
    ...
END CATCH;
```

SQL Server explicitly prohibits `TRY...CATCH` inside UDFs.

Function Cannot Return Multiple Result Sets

A stored procedure can return:

EXAMPLE / FLOW

```
Constituent result set

Gift result set

Interaction result set
```

A function cannot.

It must return:

EXAMPLE / FLOW

```
one scalar value
```

or:

EXAMPLE / FLOW

```
one table result
```

Deterministic Functions

Simple Definition

A **deterministic function** returns the same result whenever it receives the same inputs and database state relevant to that function.

Simple example:

EXAMPLE / FLOW

```
Input:
1000, 25

Calculation:
1000 - 25

Result:
975
```

Every time:

EXAMPLE / FLOW

```
same inputs
-> same output
```

Deterministic Scalar Example

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
WITH SCHEMABINDING
AS
BEGIN

    RETURN
        @TotalAmount
        -
```

```
        COALESCE(@FeeAmount, 0);

END;
GO
```

Given:

EXAMPLE / FLOW

```
1000
25
```

it always returns:

EXAMPLE / FLOW

```
975
```

Nondeterministic Functions

Simple Definition

A **nondeterministic function** can return different values even when called with the same apparent inputs.

Example built-in:

SQL

```
GETDATE()
```

Call now:

EXAMPLE / FLOW

```
2026-09-21 15:00
```

Call later:

EXAMPLE / FLOW

```
2026-09-21 15:05
```

Same call:

EXAMPLE / FLOW

```
GETDATE()
```

different output.

Therefore:

EXAMPLE / FLOW

```
GETDATE()
```

is nondeterministic.

Other Nondeterministic Examples

Examples include:

EXAMPLE / FLOW

```
GETDATE()

CURRENT_TIMESTAMP

GETUTCDATE()
```

```
NEWID()

RAND()
```

But there is an important UDF rule: SQL Server permits some nondeterministic built-ins such as `GETDATE()` in T-SQL UDFs, while side-effecting functions including `NEWID`, `NEWSEQUENTIALID`, and `RAND` are not allowed directly in a T-SQL UDF.

Determinism and Indexing

Determinism matters for features such as:

EXAMPLE / FLOW

```
Indexed computed columns

Indexed views
```

Functions used in indexed expressions must meet SQL Server determinism requirements.

Microsoft also states that a deterministic UDF must be schema-bound for SQL Server to recognize it appropriately for these purposes.

WITH SCHEMABINDING

Simple Definition

`WITH SCHEMABINDING` binds the function to database objects it references.

Example:

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_GetGiftAllocationTotal
(
    @GiftId BIGINT
)
RETURNS DECIMAL(18,2)
WITH SCHEMABINDING
AS
BEGIN

    DECLARE
        @Total DECIMAL(18,2);

    SELECT
        @Total =
            COALESCE
            (
                SUM(AllocationAmount),
                0
            )

    FROM fundraising.GiftAllocation

    WHERE GiftId =
          @GiftId;

    RETURN @Total;

END;
GO
```

What SCHEMABINDING Means

The function references:

EXAMPLE / FLOW

```
fundraising.GiftAllocation
```

SQL Server prevents incompatible changes to referenced objects that would break the schema-bound function.

For example:

EXAMPLE / FLOW

```
Dropping AllocationAmount
```

would be blocked while the schema-bound function depends on it.

Schema-bound functions must use two-part object names and referenced objects must be in the same database.

Why Use SCHEMABINDING?

Possible reasons:

EXAMPLE / FLOW

```
Protect function dependencies

Support deterministic/indexing requirements

Make object contracts clearer

Prevent accidental schema changes
```

Do not add it mechanically to every function without understanding the dependency implications.

Functions and NULL

Functions should define sensible behavior when inputs are NULL.

Example:

SQL

```
RETURN
    @TotalAmount
    -
    COALESCE
    (
        @FeeAmount,
        0
    );
```

If:

EXAMPLE / FLOW

```
@FeeAmount = NULL
```

then:

EXAMPLE / FLOW

```
COALESCE(NULL, 0)
-> 0
```

Without NULL Handling

Consider:

```sql
RETURN
    @TotalAmount
    -
    @FeeAmount;
```

If:

```
@FeeAmount = NULL
```

result becomes:

```
NULL
```

because:

```
1000 - NULL
-> NULL
```

Whether that is correct depends on the business rule.

RETURNS NULL ON NULL INPUT

SQL Server function syntax can include:

```
RETURNS NULL ON NULL INPUT
```

for certain function types/options.

Conceptually it tells SQL Server:

> If any relevant input is NULL, the function returns NULL without needing normal function evaluation.

The opposite concept is:

```
CALLED ON NULL INPUT
```

The default behavior depends on function type/options.

For most everyday T-SQL study examples, handling NULL explicitly with:

```
COALESCE

ISNULL

CASE
```

is easier to reason about.

Function Dependencies

Functions may depend on:

EXAMPLE / FLOW

```
Tables

Views

Other functions
```

Example:

EXAMPLE / FLOW

```
ufn_GetGiftAllocationTotal
        |
        v
fundraising.GiftAllocation
```

Checking Function Dependencies

SQL

```sql
SELECT
    referenced_schema_name,
    referenced_entity_name

FROM sys.sql_expression_dependencies

WHERE referencing_id =
      OBJECT_ID
      (
        'fundraising.ufn_GetGiftAllocationTotal'
      );
```

Function Metadata

Find functions:

SQL

```sql
SELECT
    SCHEMA_NAME(schema_id)
        AS SchemaName,

    name AS FunctionName,

    type_desc

FROM sys.objects

WHERE type IN
(
    'FN',
    'IF',
    'TF'
)

ORDER BY
    SchemaName,
    FunctionName;
```

Common type codes:

EXAMPLE / FLOW

```
FN
-> SQL scalar function

IF
-> Inline table-valued function

TF
-> SQL table-valued function
   typically MSTVF
```

Function Definition

SQL

```
SELECT
    OBJECT_DEFINITION
    (
        OBJECT_ID
        (
            'fundraising.ufn_ConstituentGifts'
        )
    );
```

Or:

SQL

```
EXEC sys.sp_helptext
    N'fundraising.ufn_ConstituentGifts';
```

Function Permissions

Functions are securable database objects.

For a scalar UDF, object-level permissions include:

EXAMPLE / FLOW

```
EXECUTE

REFERENCES
```

For a table-valued function, permissions include:

EXAMPLE / FLOW

```
SELECT

REFERENCES
```

and other table-like permissions according to the object permission model.

Grant EXECUTE on Scalar Function

SQL

```
GRANT EXECUTE
ON OBJECT::fundraising.ufn_CalculateNetGiftAmount
TO FundraisingAppRole;
```

Grant SELECT on TVF

```sql
GRANT SELECT
ON OBJECT::fundraising.ufn_ConstituentGifts
TO FundraisingReportingRole;
```

Permissions should follow:

```
least privilege
```

Nested Function Calls

One UDF can call another UDF.

Example:

```
ufn_CalculateNetGiftAmount
        |
        v
called by
        |
        v
another user-defined function
```

SQL Server supports nested UDF calls up to 32 levels; exceeding that limit causes the calling chain to fail.

Nested Function Example

First function:

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
AS
BEGIN

    RETURN
        @TotalAmount
        -
        COALESCE(@FeeAmount, 0);

END;
GO
```

Second function:

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftPercent
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2),
    @Percent DECIMAL(9,4)
)
RETURNS DECIMAL(18,2)
AS
BEGIN
```

```
    DECLARE
        @NetAmount DECIMAL(18,2);

    SET @NetAmount =
        fundraising.ufn_CalculateNetGiftAmount
        (
            @TotalAmount,
            @FeeAmount
        );

    RETURN
        @NetAmount
        * @Percent;

END;
GO
```

Should Functions Be Deeply Nested?

Usually avoid:

EXAMPLE / FLOW

```
Function A
   v
Function B
   v
Function C
   v
Function D
   v
Function E
```

Deep nesting can make:

EXAMPLE / FLOW

```
Performance difficult to understand

Dependencies harder to trace

Debugging harder

Query plans more complicated
```

Use reusable functions, but keep design understandable.

Built-In Functions vs User-Defined Functions

Built-In	User-Defined
Supplied by SQL Server	Created by developers
`SUM`, `DATEADD`, etc.	`ufn_ConstituentGifts`
Highly optimized system functionality	Performance depends on design
Cannot redefine	Can create/alter/drop
Huge range of categories	Scalar or table-returning custom logic

Scalar UDF vs Inline TVF

Suppose requirement:

> Return lifetime giving.

Scalar approach:

EXAMPLE / FLOW

```
one number
```

Could use scalar UDF.

Suppose requirement:

> Return all Gifts for one Constituent.

Result:

EXAMPLE / FLOW

```
multiple rows
+
multiple columns
```

Use:

EXAMPLE / FLOW

```
iTVF
```

Memory:

EXAMPLE / FLOW

```
Need one value
-> Scalar UDF

Need a relational result
-> TVF
```

Function vs View

Function	View
Can accept parameters	Cannot normally accept parameters
Scalar or table result	Table-shaped result
Scalar function can appear in expressions	View appears in `FROM`
TVF can behave like parameterized view	View reusable without arguments
Function may contain scalar logic	View represents stored SELECT

Function vs Stored Procedure

Use a function when:

EXAMPLE / FLOW

```
You need a reusable value

You need reusable table expression

You need to call it inside a query
```

```
You need parameterized relational logic
```

Use a stored procedure when:

EXAMPLE / FLOW

```
You need INSERT / UPDATE / DELETE

You need transactions

You need TRY...CATCH

You need dynamic SQL

You need temp tables

You need multiple result sets

You need workflow logic
```

Scalar Function vs Computed Expression

Before creating:

EXAMPLE / FLOW

```
ufn_IsMajorGift()
```

ask whether this simple expression is enough:

SQL

```sql
CASE
    WHEN TotalAmount >= 10000
        THEN 1
    ELSE 0
END
```

Do not create a UDF for every two-line expression.

Functions introduce:

EXAMPLE / FLOW

```
dependency

maintenance

possible performance consequences
```

Use them when reuse and abstraction justify them.

Functions and Performance

Functions are not automatically:

EXAMPLE / FLOW

```
fast
```

or:

EXAMPLE / FLOW

```
slow
```

Performance depends heavily on function type.

General preference for relational logic:

EXAMPLE / FLOW

```
Inline expression
      |
      v
Inline TVF
      |
      v
Carefully designed scalar UDF
      |
      v
MSTVF when genuinely necessary
```

This is not an absolute ranking, but it is a useful design instinct.

Important Scalar UDF Performance Issue

Microsoft notes that T-SQL UDF execution can inhibit parallel query processing when it is not successfully transformed/inlined. Scalar UDF inlining exists specifically to reduce many of the classic iterative and optimization problems of scalar functions.

Therefore check:

EXAMPLE / FLOW

```
Database compatibility level

Function eligibility

Actual execution plan

is_inlineable

Query workload
```

Important MSTVF Performance Issue

MSTVFs can have poorer cardinality information than inline relational expressions.

SQL Server 2017+ interleaved execution improves eligible cases by using actual MSTVF cardinality during optimization, but an iTVF is still often preferable when one SELECT can solve the requirement.

Function Performance Best Practices

Use these rules:

EXAMPLE / FLOW

```
Prefer set-based SQL

Prefer iTVF over MSTVF when possible

Avoid scalar UDF calls over huge row sets
unless performance is verified
```

```
Check scalar UDF inlining

Avoid wrapping indexed search columns
in functions unnecessarily

Match parameter data types to table columns

Avoid unnecessary nested UDF chains

Inspect actual execution plans

Measure logical reads and CPU

Do not assume abstraction is free
```

Practical Example 1 - Net Gift Calculation

SQL

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_CalculateNetGiftAmount
(
    @TotalAmount DECIMAL(18,2),
    @FeeAmount DECIMAL(18,2)
)
RETURNS DECIMAL(18,2)
WITH SCHEMABINDING
AS
BEGIN

    RETURN
        @TotalAmount
        -
        COALESCE(@FeeAmount, 0);

END;
GO
```

Use:

SQL

```sql
SELECT
    GiftId,
    TotalAmount,

    fundraising.ufn_CalculateNetGiftAmount
    (
        TotalAmount,
        25
    ) AS NetAmount

FROM fundraising.Gift;
```

Practical Example 2 - Constituent Gifts iTVF

SQL

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGifts
(
    @ConstituentId BIGINT
```

```
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        CampaignId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId
);
GO
```

Use:

```SQL
SELECT *
FROM fundraising.ufn_ConstituentGifts
(
    1001
);
```

Practical Example 3 - Campaign Gifts

```SQL
CREATE OR ALTER FUNCTION fundraising.ufn_CampaignGifts
(
    @CampaignId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        ConstituentId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE CampaignId =
          @CampaignId
);
GO
```

Call:

```SQL
SELECT *
FROM fundraising.ufn_CampaignGifts
(
    200
);
```

Practical Example 4 - Fund Allocations

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_FundAllocations
(
    @FundId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        ga.GiftAllocationId,
        ga.GiftId,
        ga.FundId,
        ga.AllocationAmount

    FROM fundraising.GiftAllocation AS ga

    WHERE ga.FundId =
          @FundId
);
GO
```

Practical Example 5 - Constituents and TVF with APPLY

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    g.GiftId,
    g.GiftDate,
    g.TotalAmount

FROM constituent.Constituent AS c

OUTER APPLY
    fundraising.ufn_ConstituentGifts
    (
        c.ConstituentId
    ) AS g;
```

This keeps:

EXAMPLE / FLOW

```
donors
and
Constituents without Gifts
```

because **OUTER APPLY** preserves the left row.

Practical Example 6 - Gift Allocation Total Scalar Function

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_GetGiftAllocationTotal
(
    @GiftId BIGINT
)
RETURNS DECIMAL(18,2)
AS
BEGIN

    DECLARE
```

```
        @Amount DECIMAL(18,2);

    SELECT
        @Amount =
            COALESCE
            (
                SUM(AllocationAmount),
                0
            )

    FROM fundraising.GiftAllocation

    WHERE GiftId =
          @GiftId;

    RETURN @Amount;

END;
GO
```

Use:

SQL

```
SELECT
    GiftId,
    TotalAmount,

    fundraising.ufn_GetGiftAllocationTotal
    (
        GiftId
    ) AS AllocatedAmount

FROM fundraising.Gift;
```

Performance Rewrite of the Previous Example

For a large report, this may be better:

SQL

```
SELECT
    g.GiftId,
    g.TotalAmount,

    COALESCE
    (
        SUM(ga.AllocationAmount),
        0
    ) AS AllocatedAmount

FROM fundraising.Gift AS g

LEFT JOIN fundraising.GiftAllocation AS ga
    ON ga.GiftId =
       g.GiftId

GROUP BY
    g.GiftId,
    g.TotalAmount;
```

Reason:

EXAMPLE / FLOW

```
One set-based aggregation
```

may be easier for SQL Server to optimize than repeated scalar UDF execution.

Practical Example 7 - Giving Summary iTVF

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGivingSummary
(
    @ConstituentId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        ConstituentId,

        COUNT_BIG(*)
            AS GiftCount,

        SUM(TotalAmount)
            AS LifetimeGiving,

        AVG(TotalAmount)
            AS AverageGift,

        MAX(TotalAmount)
            AS LargestGift,

        MAX(GiftDate)
            AS LatestGiftDate

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId

    GROUP BY ConstituentId
);
GO
```

Use:

SQL

```
SELECT *
FROM fundraising.ufn_ConstituentGivingSummary
(
    1001
);
```

Practical Example 8 - Use iTVF with Constituent

SQL

```
SELECT
    c.ConstituentId,
    c.DisplayName,

    s.GiftCount,
    s.LifetimeGiving,
    s.LargestGift

FROM constituent.Constituent AS c

OUTER APPLY
    fundraising.ufn_ConstituentGivingSummary
    (
```

```
        c.ConstituentId
    ) AS s;
```

Common Function Mistakes

Mistake 1 - Using a Function for Data Modification

Wrong design:

EXAMPLE / FLOW

```
ufn_CreateGift()
```

A UDF is not the right tool for inserting Gifts.

Use:

EXAMPLE / FLOW

```
usp_CreateGift
```

stored procedure.

Mistake 2 - Using Scalar UDF Everywhere

Do not turn every expression into:

EXAMPLE / FLOW

```
dbo.ufn_Something(...)
```

especially in large table scans.

Consider:

EXAMPLE / FLOW

```
CASE

JOIN

CTE

derived table

iTVF
```

first.

Mistake 3 - Using MSTVF When One SELECT Is Enough

If this works:

SQL

```
RETURNS TABLE
AS
RETURN
(
    SELECT ...
)
```

do not unnecessarily create a return table variable with multiple statements.

Mistake 4 - Calling Scalar UDF Without Schema

Avoid:

```sql
SELECT ufn_CalculateNetGiftAmount(...);
```

Use:

```sql
SELECT
    fundraising.ufn_CalculateNetGiftAmount(...);
```

Mistake 5 - Function Around Indexed WHERE Column

Be careful with:

```sql
WHERE dbo.Function(GiftDate) = ...
```

This can hurt SARGability and index access.

Mistake 6 - Assuming SQL Server Will Inline Every Scalar UDF

It will not.

Eligibility and compilation context matter.

Check:

```
is_inlineable

actual execution plan
```

Mistake 7 - Ignoring NULL Behavior

Decide whether:

```
NULL fee
```

means:

```
zero fee
```

or:

```
unknown fee
```

Then code accordingly.

Mistake 8 - Trying Dynamic SQL in a Function

Not allowed.

Use:

```
Stored procedure
```

when SQL structure must vary dynamically.

Mistake 9 - Trying #Temp Tables in UDF

Not allowed.

Use:

EXAMPLE / FLOW

```
Table variable
```

for MSTVF-local table storage.

Mistake 10 - Deep Function Nesting

Avoid large dependency chains.

They become harder to understand and tune.

Choosing the Correct Function Type

Ask:

EXAMPLE / FLOW

```
Do I need one scalar value?
```

Use:

EXAMPLE / FLOW

```
Scalar UDF
```

Ask:

EXAMPLE / FLOW

```
Do I need rows/columns
and one SELECT can produce them?
```

Use:

EXAMPLE / FLOW

```
Inline TVF
```

Ask:

EXAMPLE / FLOW

```
Do I truly need multiple statements
to build a table result?
```

Consider:

EXAMPLE / FLOW

```
MSTVF
```

Ask:

EXAMPLE / FLOW

```
Do I need to modify data,
use transactions,
dynamic SQL,
temp tables,
or error handling?
```

Use:

EXAMPLE / FLOW

```
Stored Procedure
```

Master Function Comparison

Requirement	Best Starting Choice
Calculate one reusable value	Scalar UDF
Return parameterized rows	iTVF
Return multi-step table result	MSTVF
Reusable parameterless table query	View
Modify data	Stored Procedure
Transactional workflow	Stored Procedure
Dynamic SQL	Stored Procedure
Temporary query-only organization	CTE
Built-in calculation	Built-in Function

25 Interview Questions and Practical Problems

1. What is a function in SQL Server?

A function is reusable SQL logic that returns either:

EXAMPLE / FLOW

```
one scalar value
```

or:

EXAMPLE / FLOW

```
a table.
```

Functions can be called from other SQL queries.

2. Why use functions?

Functions are useful for:

EXAMPLE / FLOW

```
Reusable calculations

Parameterized table queries

Centralizing repeated query logic

Using reusable expressions inside SELECT/WHERE

Returning relational results for JOIN/APPLY
```

3. What are the main SQL Server function types?

At a practical application-development level:

EXAMPLE / FLOW

```
Built-in functions

User-defined scalar functions

Inline table-valued functions

Multi-statement table-valued functions
```

4. What is a scalar-valued function?

A scalar function returns one value.

Example:

SQL

```sql
SELECT
    fundraising.ufn_CalculateNetGiftAmount
    (
        1000,
        25
    );
```

returns:

EXAMPLE / FLOW

```
975
```

5. What is an inline table-valued function?

An iTVF returns a table produced from one SELECT.

Example:

SQL

```sql
SELECT *
FROM fundraising.ufn_ConstituentGifts
(
    1001
);
```

returns Gifts for Constituent 1001.

6. What is a multi-statement TVF?

An MSTVF:

EXAMPLE / FLOW

```
defines a return table variable,

executes multiple statements,

populates that table,

and returns it.
```

Use it only when the result genuinely needs multi-step processing.

7. What is the difference between iTVF and MSTVF?

EXAMPLE / FLOW

```
iTVF
-> One SELECT
-> Optimizer-friendly

MSTVF
-> Multiple statements
-> Return table variable
-> More performance concerns
```

8. What is the difference between a function and a stored procedure?

A function returns a scalar value or table and can participate directly in SQL queries.

A procedure is executed with **EXEC** and can perform wider operations such as:

EXAMPLE / FLOW

```
INSERT

UPDATE

DELETE

transactions

dynamic SQL

TRY...CATCH
```

9. Can a function modify database tables?

A normal T-SQL UDF cannot modify persistent database state.

It cannot use ordinary INSERT/UPDATE/DELETE against application tables.

10. Can a function use temporary tables?

No.

T-SQL UDFs cannot use `#temp` tables.

Table variables are permitted.

11. Can a function use dynamic SQL?

No.

T-SQL UDFs cannot execute dynamic SQL.

If dynamic SQL is required, use a stored procedure.

12. Can a function use TRY...CATCH?

No.

`TRY...CATCH` is not supported inside a T-SQL UDF.

13. What is a deterministic function?

A deterministic function returns the same result for the same inputs and relevant database state.

Example:

EXAMPLE / FLOW

```
1000 - 25
-> always 975
```

14. What is a nondeterministic function?

A nondeterministic function can return different results across calls.

Example:

SQL

```
GETDATE()
```

because time changes.

15. What is SCHEMABINDING?

`WITH SCHEMABINDING` binds the function to referenced database objects so incompatible schema changes cannot break the function silently.

It is also important for determinism-related indexing scenarios.

16. Can functions call other functions?

Yes.

T-SQL UDFs may be nested up to 32 levels.

Avoid excessive nesting because it complicates maintenance and performance analysis.

17. What is scalar UDF inlining?

Starting with SQL Server 2019, eligible scalar UDFs can be transformed into relational expressions inside the calling query.

This allows SQL Server's optimizer to optimize the function logic together with the outer query and can significantly reduce traditional scalar UDF overhead.

18. Does every scalar UDF get inlined?

No.

Eligibility depends on the function definition and usage context.

You can inspect:

EXAMPLE / FLOW

```
sys.sql_modules.is_inlineable
```

but even an eligible function is not guaranteed to be inlined in every compilation.

19. Why are inline TVFs often preferred over MSTVFs?

An iTVF exposes its single relational expression directly to the optimizer.

An MSTVF builds rows inside a return table variable and has historically had cardinality-estimation challenges.

Modern SQL Server can use interleaved execution for eligible MSTVFs, but iTVFs are still often preferable when one SELECT can solve the requirement.

20. What is the difference between a TVF and a view?

Both can produce table-shaped results.

The key difference is:

EXAMPLE / FLOW

```
View
-> No ordinary parameters

TVF
-> Can accept parameters
```

An iTVF is often described conceptually as a parameterized view.

21. Query Problem - Create a function returning one Constituent's Gifts.

SQL

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentGifts
(
    @ConstituentId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        GiftId,
        CampaignId,
        GiftDate,
        TotalAmount

    FROM fundraising.Gift

    WHERE ConstituentId =
          @ConstituentId
);
```

Use:

SQL

```sql
SELECT *
FROM fundraising.ufn_ConstituentGifts
(
    1001
);
```

This is an inline table-valued function.

22. Query Problem - Use a TVF for every Constituent.

SQL

```sql
SELECT
    c.ConstituentId,
    c.DisplayName,

    g.GiftId,
    g.TotalAmount

FROM constituent.Constituent AS c

OUTER APPLY
    fundraising.ufn_ConstituentGifts
    (
        c.ConstituentId
    ) AS g;
```

`OUTER APPLY` keeps Constituents even when their TVF call returns no Gifts.

23. Query Problem - Return total allocations for a Gift.

Scalar version:

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_GetGiftAllocationTotal
(
    @GiftId BIGINT
)
RETURNS DECIMAL(18,2)
AS
BEGIN

    DECLARE
        @Total DECIMAL(18,2);

    SELECT
        @Total =
            COALESCE
            (
                SUM(AllocationAmount),
                0
            )

    FROM fundraising.GiftAllocation

    WHERE GiftId =
          @GiftId;

    RETURN @Total;

END;
```

Then:

```sql
SELECT
    GiftId,
    TotalAmount,

    fundraising.ufn_GetGiftAllocationTotal
    (
        GiftId
    ) AS AllocationTotal

FROM fundraising.Gift;
```

For a large report, compare this with a normal set-based `JOIN + GROUP BY` version because the set-based form may perform better.

24. Query Problem - Build a Fund allocation TVF.

```sql
CREATE OR ALTER FUNCTION fundraising.ufn_FundAllocations
(
    @FundId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
```

```
        ga.GiftAllocationId,
        ga.GiftId,
        ga.FundId,
        ga.AllocationAmount

    FROM fundraising.GiftAllocation AS ga

    WHERE ga.FundId =
          @FundId
);
```

Usage:

SQL

```
SELECT *
FROM fundraising.ufn_FundAllocations
(
    300
);
```

This is a good iTVF because the requirement is naturally expressed by one SELECT.

25. Full Interview Problem

Requirement

Create reusable functionality that:

EXAMPLE / FLOW

```
Accepts a ConstituentId

Returns GiftId

GiftDate

CampaignName

TotalAmount

FundName

AllocationAmount
```

One SELECT can solve this, so use an:

EXAMPLE / FLOW

```
Inline TVF
```

Solution:

SQL

```
CREATE OR ALTER FUNCTION fundraising.ufn_ConstituentFundraisingHistory
(
    @ConstituentId BIGINT
)
RETURNS TABLE
AS
RETURN
(
    SELECT
        g.GiftId,
        g.GiftDate,
        g.TotalAmount,
```

```
        ca.CampaignId,
        ca.CampaignName,

        f.FundId,
        f.FundName,

        ga.AllocationAmount

    FROM fundraising.Gift AS g

    LEFT JOIN fundraising.Campaign AS ca
        ON ca.CampaignId =
           g.CampaignId

    INNER JOIN fundraising.GiftAllocation AS ga
        ON ga.GiftId =
           g.GiftId

    INNER JOIN fundraising.Fund AS f
        ON f.FundId =
           ga.FundId

    WHERE g.ConstituentId =
          @ConstituentId
);
GO
```

Call:

SQL

```
SELECT
    GiftId,
    GiftDate,
    CampaignName,
    FundName,
    TotalAmount,
    AllocationAmount

FROM fundraising.ufn_ConstituentFundraisingHistory
(
    1001
)

ORDER BY
    GiftDate DESC,
    GiftId DESC;
```

Understand the Full Example

Input:

EXAMPLE / FLOW

```
ConstituentId = 1001
```

Function:

EXAMPLE / FLOW

```
Gift
 |
 +---- Campaign
 |
```

```
 v
GiftAllocation
 |
 v
Fund
```

Result:

EXAMPLE / FLOW

```
All fundraising history
for that Constituent
```

This is a good function because:

EXAMPLE / FLOW

```
It returns a relational result

It accepts a parameter

One SELECT can express it

It does not modify data

It can participate in JOIN/APPLY queries
```

Therefore:

EXAMPLE / FLOW

```
Inline TVF
```

is a natural design.

Final Revision Map

EXAMPLE / FLOW

```
SQL SERVER FUNCTIONS
        |
        +-----------------------------+
        |                             |
   BUILT-IN                      USER-DEFINED
        |                             |
 SUM / AVG / DATEADD                 +---- Scalar UDF
 GETDATE / UPPER                     |
 ROW_NUMBER / etc.                   +---- Inline TVF
                                      |
                                      +---- Multi-Statement TVF
```

Scalar:

EXAMPLE / FLOW

```
Inputs
  |
  v
Scalar UDF
  |
  v
One Value
```

Inline TVF:

EXAMPLE / FLOW

```
Parameters
   |
   v
One SELECT
   |
   v
Table Result
```

MSTVF:

EXAMPLE / FLOW

```
Parameters
   |
   v
Return Table Variable
   |
   +---- Statement 1
   +---- Statement 2
   +---- Statement 3
   |
   v
Table Result
```

Function choice:

EXAMPLE / FLOW

```
One reusable value
-> Scalar UDF

One SELECT returning rows
-> Inline TVF

Multiple statements needed
to construct table
-> MSTVF

Modify data / transaction /
dynamic SQL / TRY...CATCH
-> Stored Procedure
```

Final Easy Memory Rules

EXAMPLE / FLOW

```
Function
-> Accept input and return something
```

EXAMPLE / FLOW

```
Scalar UDF
-> One value
```

EXAMPLE / FLOW

```
Inline TVF
-> One SELECT returns a table
```

EXAMPLE / FLOW

```
MSTVF
-> Multiple statements build a table
```

EXAMPLE / FLOW

```
View
-> Table query without normal parameters
```

EXAMPLE / FLOW

```
iTVF
-> Parameterized view-like query
```

EXAMPLE / FLOW

```
Stored Procedure
-> Perform database operation/workflow
```

EXAMPLE / FLOW

```
Deterministic
-> Same inputs, same result
```

EXAMPLE / FLOW

```
Nondeterministic
-> Result can change
```

EXAMPLE / FLOW

```
SCHEMABINDING
-> Protect referenced schema dependencies
```

EXAMPLE / FLOW

```
Scalar UDF Inlining
-> Optimizer may transform eligible scalar UDF logic
  into the calling query
```

EXAMPLE / FLOW

```
CROSS APPLY
-> Keep left row only when TVF returns rows
```

EXAMPLE / FLOW

```
OUTER APPLY
-> Keep left row even if TVF returns no rows
```

EXAMPLE / FLOW

```
Performance rule
-> Prefer set-based logic and iTVFs
  when they naturally solve the requirement
```

The most important rule is:

Choose the function type from the shape of the result. If you need one reusable value, consider a scalar UDF. If you need parameterized rows and one SELECT can return them, prefer an inline TVF. Use a multi-statement TVF only when multiple statements are genuinely necessary. If the operation modifies database state, requires transactions, temp tables, dynamic SQL, or error handling, use a stored procedure instead.